Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas de la Información
Boca del Monte, Villa Canales
Programación I
Sección B

REPORTE DE AVANCE NO.1

PROYECTO (TOTITO 3 EN RAYA MATRIZ 10 X 10)

ESTUDIANTE	**CARNÉ**
Bryan Alejandro Osoy Monterroso	7690-25-15671

INTRODUCCION

El presente documento describe el primer avance del Proyecto 01, correspondiente al curso de Programación 1. El proyecto consiste en la implementación del juego Totito (3 en raya) sobre una matriz de 10×10 utilizando el lenguaje Java y aplicando Programación Orientada a Objetos.

En esta primera etapa del desarrollo se trabajó principalmente en definir la estructura del proyecto, identificar las clases necesarias y comenzar a codificar las clases base que dan soporte al juego: la clase Jugador y la clase Tablero.

AVANCE CODIFICACION

- **Clases Completadas:**
  Jugador.java: 100%
  Tablero.java: 100%
  Partida.java: "pendiente"
  Main.java: "pendiente"

CLASE JUGADOR : Esta fue la primera clase en desarrollarse por ser la más sencilla y servir de base para las demás. La clase Jugador recopila toda la información de un participante del juego.

Se definieron tres atributos privados: nombre (String) para almacenar el nombre del jugador, simbolo (char) para el carácter que usará en el tablero, y puntaje (int) para llevar el conteo de partidas ganadas. Se implementaron los métodos get y set para cada atributo siguiendo el principio de encapsulamiento de la POO, además del método sumarPunto() que incrementa el puntaje y toString() para mostrar la información del jugador.

CLASE TABLERO: Esta fue la clase más compleja de esta primera etapa. Maneja la matriz bidimensional de 10×10 que representa el área de juego.

Se comenzó implementando el constructor que inicializa la matriz con el carácter '.' en todas sus posiciones para indicar que están vacías. Luego se desarrolló el método mostrar Tablero() que imprime la cuadrícula con los números de fila y columna para que los jugadores puedan orientarse al hacer sus jugadas.

El método más complicado fue hay Ganador(), el cual revisa las cuatro direcciones posibles de victoria: horizontal, vertical, diagonal descendente y diagonal ascendente. Se utilizaron bucles for anidados para recorrer todas las posiciones posibles en cada dirección. Fue necesario hacer varias pruebas antes de codificarlo para asegurarse de que los índices fueran correctos.

También se implementó colocarSimbolo() con validación de rango y de casilla libre, y tableroLleno() para detectar empates.

DIFICULTADES ENCONTRADAS

La principal dificultad en esta etapa fue el diseño del algoritmo hayGanador() que tomo mas de 2 dias. Al principio era confuso manejar los índices correctamente para las diagonales, especialmente la diagonal ascendente , ya que el índice de columna disminuye en lugar de aumentar. Se tuvo que dibujar la matriz en papel y probar manualmente con ejemplos antes de lograr que el código fuera correcto.

Otra dificultad menor fue la alineación del tablero al imprimirlo en consola, ya que el número 10 tiene dos dígitos y desalineaba la cuadrícula. Se resolvió con una condición que agrega un espacio extra antes de los números de una sola cifra.

CAPTURAS DE PANTALLA DE CODIGOS

JUGADOR:

```java
// Clase que representa a un jugador del juego
// Autor: [Bryan Alejandro Osoy Monterroso] - Carnet: [7690 25 15671]

public class Jugador {

    private String nombre;
    private char simbolo;
    private int puntaje;

    // Constructor del jugador
    public Jugador(String nombre, char simbolo) {
        this.nombre = nombre;
        this.simbolo = simbolo;
        this.puntaje = 0;
    }

    // metodos get y set
    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public char getSimbolo() {
        return simbolo;
    }

    public void setSimbolo(char simbolo) {
        this.simbolo = simbolo;
    }

    public int getPuntaje() {
        return puntaje;
    }

    public void sumarPunto() {
        this.puntaje++;
    }

    // para mostrar info del jugador
    public String toString() {
        return nombre + " (" + simbolo + ") - Puntos: " + puntaje;
    }
}

```

TABLERO:

```java
// Clase que maneja el tablero de 10x10 y la logica del juego
// Autor: [Bryan Alejandro Osoy Monterroso] - Carnet: [7690 25 15671]

public class Tablero {

    // la matriz del juego es de 10x10
    private char[][] matriz;
    private int filas = 10;
    private int columnas = 10;

    public Tablero() {
        matriz = new char[filas][columnas];
        limpiarTablero();
    }

    // limpia el tablero poniendo espacios vacios
    public void limpiarTablero() {
        for (int i = 0; i < filas; i++) {
            for (int j = 0; j < columnas; j++) {
                matriz[i][j] = '.';
            }
        }
    }

    // muestra el tablero en consola
    public void mostrarTablero(String nombreJ1, String carnetJ1) {
        System.out.println("\n   1 2 3 4 5 6 7 8 9 10");
        for (int i = 0; i < filas; i++) {
            // para alinear bien los numeros de fila
            if (i < 9) {
                System.out.print((i + 1) + "  ");
            } else {
                System.out.print((i + 1) + " ");
            }
            for (int j = 0; j < columnas; j++) {
                System.out.print(matriz[i][j] + " ");
            }
            System.out.println();
        }
        // mostrar nombre y carnet debajo del tablero
        System.out.println("\n" + nombreJ1 + " - " + carnetJ1);
    }

    // coloca el simbolo en la posicion indicada
    public boolean colocarSimbolo(int fila, int columna, char simbolo) {
        // verificamos que la casilla este libre
        if (fila >= 0 && fila < filas && columna >= 0 && columna < columnas) {
            if (matriz[fila][columna] == '.') {
                matriz[fila][columna] = simbolo;
                return true;
            }
        }
        return false;
    }

    // revisa si alguien gano
    public boolean hayGanador(char simbolo) {
        // revisar filas
        for (int i = 0; i < filas; i++) {
            for (int j = 0; j <= columnas - 3; j++) {
                if (matriz[i][j] == simbolo && matriz[i][j+1] == simbolo && matriz[i][j+2] == simbolo) {
                    return true;
                }
            }
        }

```

```java
// revisar columnas
        for (int i = 0; i <= filas - 3; i++) {
            for (int j = 0; j < columnas; j++) {
                if (matriz[i][j] == simbolo && matriz[i+1][j] == simbolo && matriz[i+2][j] == simbolo) {
                    return true;
                }
            }
        }

        // revisar diagonal hacia abajo-derecha
        for (int i = 0; i <= filas - 3; i++) {
            for (int j = 0; j <= columnas - 3; j++) {
                if (matriz[i][j] == simbolo && matriz[i+1][j+1] == simbolo && matriz[i+2][j+2] == simbolo) {
                    return true;
                }
            }
        }

        // revisar diagonal hacia abajo-izquierda
        for (int i = 0; i <= filas - 3; i++) {
            for (int j = 2; j < columnas; j++) {
                if (matriz[i][j] == simbolo && matriz[i+1][j-1] == simbolo && matriz[i+2][j-2] == simbolo) {
                    return true;
                }
            }
        }

        return false;
    }

    // revisa si el tablero esta lleno (empate)
    public boolean tableroLleno() {
        for (int i = 0; i < filas; i++) {
            for (int j = 0; j < columnas; j++) {
                if (matriz[i][j] == '.') {
                    return false;
                }
            }
        }
        return true;
    }

    // devuelve la matriz para poder guardarla
    public char[][] getMatriz() {
        return matriz;
    }

    // carga una matriz guardada
    public void setMatriz(char[][] nuevaMatriz) {
        this.matriz = nuevaMatriz;
    }
}
```

CONCLUSION DEL AVANCE

En esta primera etapa se logró establecer la base del proyecto con las dos clases más importantes desde el punto de vista de la lógica del juego. La clase Jugador provee la identidad de cada participante y la clase Tablero maneja todo el estado del área de juego, incluyendo la detección de victorias.

El avance es satisfactorio y se tiene confianza en poder completar las clases restantes en los dos días siguientes ya que lleva una semana trabando para tener el sistema funcionando de forma completa.